

# FUT MANAGER

## Simulateur de match 2D - V0.9

Rapport technique exhaustif, bilan d'implémentation et document de reprise

**VERSION DE RÉFÉRENCE :** V0.9. La V0.10 expérimentale a été explicitement abandonnée et ne doit pas être considérée comme une évolution valide de la baseline.

**OBJECTIF DU DOCUMENT :** Permettre à un développeur ou à un LLM sans contexte conversationnel de reprendre le projet à partir du code V0.9, du Game Design initial et de ce rapport, sans réinventer les choix déjà stabilisés.

**PRINCIPE D'ARCHITECTURE :** Le moteur de simulation est un module serveur déterministe indépendant de l'affichage. Le front ne décide jamais du résultat du match : il rejoue un replay déjà calculé.

État de synthèse : 21 juillet 2026

# Sommaire

- 1. Résumé exécutif et état actuel
- 2. Vision du jeu et objectifs du simulateur
- 3. Architecture générale et modularité
- 4. Données joueurs et modèle de statistiques
- 5. Contrat d'entrée/sortie du moteur
- 6. Temps, seed, déterminisme et boucle de simulation
- 7. Modèle des agents joueurs
- 8. Intelligence artificielle utilitaire et température
- 9. Positionnement collectif, formation, bloc et appels
- 10. Rôles, tactiques, synergies et hors-poste
- 11. Mouvement, fatigue, duels et états temporaires
- 12. Physique du ballon, passes et contrôles
- 13. Tirs, gardiens, buts et rebonds
- 14. Règles de football et coups de pied arrêtés
- 15. Hors-jeu, fautes, cartons, blessures et remplacements
- 16. Replay et viewer 2D
- 17. Interface actuelle
- 18. Balance Lab, Monte-Carlo, heatmaps et micro-benchmarks
- 19. État quantitatif observé en V0.9
- 20. Ce qui est solide / ce qui reste fragile
- 21. Limites connues et dette technique
- 22. Architecture modulaire recommandée pour la suite
- 23. Prochaine étape recommandée : équipe et tactiques
- 24. Roadmap produit complète
- 25. Idées d'évolution du moteur à plus long terme
- 26. Stratégie de tests et qualité
- 27. Handoff à un futur LLM / développeur
- Annexe A. Fichiers importants de la V0.9
- Annexe B. Invariants à ne pas casser
- Annexe C. Paramètres principaux de configuration

# 1. Résumé exécutif et état actuel

Le projet dispose aujourd'hui d'un simulateur de football 2D fonctionnel, déterministe et suffisamment modulaire pour servir de socle à un jeu navigateur de management/collection. La simulation est intégralement calculée côté serveur. Le navigateur reçoit ensuite le résultat, les statistiques, les événements et un replay temporel qu'il interpole visuellement.

Le moteur V0.9 ne tire pas un score final en amont. Le score émerge de décisions individuelles, de déplacements, de trajectoires physiques du ballon, de contrôles, d'interceptions, de tirs, d'arrêts, de sorties de balle et de coups de pied arrêtés.

| Bloc                                          | État V0.9                                             |
|-----------------------------------------------|-------------------------------------------------------|
| Moteur headless déterministe                  | Implémenté                                            |
| IA individuelle / agents joueurs              | Implémentée, utility AI + softmax                     |
| Physique de balle                             | Implémentée : vitesse, décélération, contrôle, rebond |
| Règles de match                               | Implémentées en version simplifiée                    |
| Replay serveur -> client                      | Implémenté                                            |
| Viewer Canvas 2D                              | Implémenté                                            |
| Fatigue / blessures / cartons / changements   | Implémentés                                           |
| Formation 4-3-3                               | Implémentée                                           |
| Rôles individuels                             | Implémentés                                           |
| Tactiques collectives                         | Implémentées mais limitées                            |
| Synergie par nationalité locale               | Implémentée                                           |
| Analytics Monte-Carlo / heatmaps              | Implémentés                                           |
| Micro-benchmarks isolés                       | Implémentés                                           |
| Écran de construction d'équipe                | À faire                                               |
| Multiples formations                          | À faire                                               |
| Persistence complète / collection / PvP async | À faire                                               |

**Décision importante :** le moteur est désormais assez mature pour être gelé temporairement. La priorité produit peut passer à la construction d'équipe et aux stratégies, en conservant le simulateur comme module indépendant.

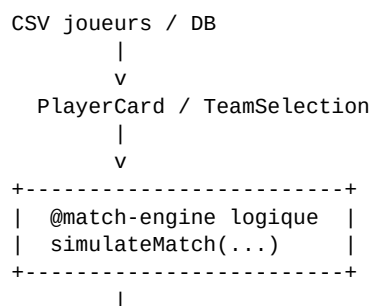
# 2. Vision du jeu et objectifs du simulateur

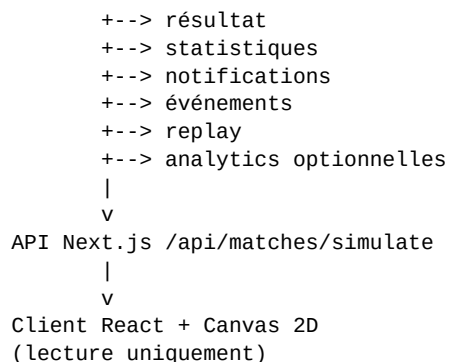
Le jeu visé est un manager de football volontairement simple : le joueur prépare son équipe et sa tactique avant le match, puis n'intervient plus pendant la rencontre. Il peut regarder le match, accélérer le replay ou passer directement au résultat.

- Collection permanente de joueurs, inspirée de FUT.
- Préparation tactique inspirée de Football Manager mais très simplifiée.
- PvP asynchrone et bots à terme.
- Aucune intervention manuelle pendant le match.
- Une simulation assez lisible pour comprendre l'effet des joueurs, rôles et tactiques.
- Un moteur peu coûteux, déterministe, testable en batch et indépendant de l'interface.

Le simulateur a donc deux responsabilités : produire un résultat crédible à partir de deux snapshots tactiques, puis produire suffisamment de données pour rendre ce résultat visible et explicable.

# 3. Architecture générale et modularité





Cette séparation est l'un des choix les plus importants du projet. Le Canvas ne contient aucune logique qui décide d'un but, d'une passe ou d'un duel. Le serveur peut simuler un match sans navigateur, et le Balance Lab peut lancer des centaines de matchs avec `recordReplay=false`.

| Module actuel                     | Responsabilité                                                 |
|-----------------------------------|----------------------------------------------------------------|
| lib/game/engine.ts                | Boucle de match, agents, règles, physique, événements, replay. |
| lib/game/config.ts                | Paramètres centralisés de game design et de simulation.        |
| lib/game/types.ts                 | Contrats de données partagés.                                  |
| lib/game/formations.ts            | 4-3-3, slots, ancrs, voisinages, repère d'attaque.             |
| lib/game/compatibility.ts         | Hors-poste, fatigue et stats effectives.                       |
| lib/game/rng.ts                   | PRNG seedé déterministe.                                       |
| lib/data/load-players.ts          | Chargement du CSV normalisé.                                   |
| app/api/matches/simulate          | Frontière HTTP de simulation.                                  |
| components/PitchCanvas.tsx        | Lecture et interpolation du replay.                            |
| app/api/analytics/monte-carlo     | Batch headless, sensibilité et agrégation.                     |
| lib/analytics/micro-benchmarks.ts | Tests isolés des six stats.                                    |

## 4. Données joueurs et modèle de statistiques

Le simulateur consomme un format joueur canonique indépendant du CSV FIFA d'origine. Le pipeline Python développé séparément transforme le dataset FC 26 en une base/CSV normalisé.

| Champ                  | Usage                                                               |
|------------------------|---------------------------------------------------------------------|
| playerId               | Identité stable                                                     |
| shortName / longName   | Affichage                                                           |
| nationalityName        | Synergie                                                            |
| primaryPosition        | Poste naturel principal                                             |
| alternativePositions[] | Postes naturels secondaires                                         |
| overall / potential    | Informations de collection, pas utilisées directement par le moteur |
| speed                  | Déplacement et poursuites                                           |
| shooting               | Précision/exécution des tirs                                        |
| passing                | Précision/dosage des passes et contrôle                             |
| physical               | Duels, fatigue et première touche                                   |
| technique              | Contrôle, dribble, exécution                                        |
| intelligence           | Qualité de décision, placement et lecture du jeu                    |

Les six statistiques vont de 1 à 100. Le moteur travaille ensuite sur des stats effectives modifiées par la fatigue, la compatibilité de poste et la synergie.

## 5. Contrat d'entrée/sortie du moteur

```

simulateMatch({
  home: TeamSelection,
  away: TeamSelection,
  players: PlayerCard[],
  seed: string,
  logicalSeconds?: number,
  recordReplay?: boolean,

```

```

    recordSpatialAnalytics?: boolean
  }) -> MatchSimulationOutput

```

- Entrée : deux équipes, leurs joueurs, rôles, tactiques et une seed.
- Sortie : score, stats, blessures/suspensions, replay et éventuellement analytics spatiales.
- recordReplay=false permet le calcul massif sans enregistrer les frames Canvas.
- recordSpatialAnalytics=true active les heatmaps et métriques de bloc.

À terme, ce contrat doit devenir une interface stable et versionnée. Le reste du jeu ne devrait dépendre que de ce contrat, pas des structures internes de RuntimePlayer ou MatchState.

## 6. Temps, seed, déterminisme et boucle de simulation

| Paramètre V0.9                  | Valeur                     |
|---------------------------------|----------------------------|
| Temps affiché réglementaire     | 90 minutes                 |
| Temps logique réglementaire     | 360 secondes par défaut    |
| Pas principal physique          | 0,2 s                      |
| Sous-pas ballon                 | 0,04 s                     |
| Décision IA                     | 1 fois par seconde logique |
| Snapshot replay                 | toutes les 0,16 s          |
| Échantillon analytics spatiales | 1 s logique                |

La seed initialise un PRNG déterministe. À entrée identique, configuration identique et seed identique, le moteur doit reproduire le même match. C'est indispensable pour le debug, les replays et les expériences appariées du Balance Lab.

Le match est organisé en deux périodes. Chaque période possède un temps réglementaire logique et un temps additionnel calculé à partir des arrêts de jeu, avec un ratio de compensation configurable.

**Invariant** : ne jamais utiliser Math.random(), Date.now() ou une horloge système dans le coeur du moteur. Toute stochasticité de gameplay doit passer par SeededRng.

## 7. Modèle des agents joueurs

Chaque joueur actif est un agent léger. Ce ne sont pas des agents LLM : il s'agit d'agents de simulation classiques, avec état local, perception du monde, génération d'actions candidates, fonction d'utilité et choix stochastique contrôlé.

| État runtime de l'agent                  | Signification                                           |
|------------------------------------------|---------------------------------------------------------|
| pos / target                             | Position actuelle et cible de déplacement               |
| slotId / assignedPosition                | Slot de formation et poste joué                         |
| role                                     | DEFENSIVE / NORMAL / OFFENSIVE / CREATOR / PRESSING     |
| energy                                   | Fatigue intra-match                                     |
| synergyBonus                             | Bonus Intelligence issu des voisins de même nationalité |
| stunnedUntil                             | Déséquilibre temporaire après duel perdu                |
| runTarget / runUntil                     | Intention persistante d'appel sans ballon               |
| yellowCards / redCard / injured / active | État réglementaire                                      |

Le joueur n'exécute pas une machine à états rigide par poste. Son poste définit surtout son ancre et sa zone de responsabilité ; le moteur générique utilise ensuite le contexte, le rôle et les statistiques pour adapter ses décisions.

## 8. Intelligence artificielle utilitaire et température

Le coeur décisionnel est une utility AI. Le porteur évalue actuellement PASS, SHOT, DRIBBLE et HOLD. Chaque candidat reçoit une utility à partir du contexte : progression, espace, sécurité de ligne de passe, pression, tactique collective, rôle, hors-jeu, appels en cours, etc.

1. Percevoir le contexte
2. Générer les actions plausibles
3. Calculer U(action)

4. Calculer la température depuis Intelligence
5. Échantillonner par softmax
6. Résoudre physiquement l'action choisie

L'Intelligence n'est donc pas une simple probabilité de réussite. Elle réduit la température de décision : un joueur intelligent concentre davantage sa probabilité sur les meilleures utilities ; un joueur moins intelligent sélectionne plus souvent une option sous-optimale.

$$T = T_{\min} + (1 - \text{Intelligence}/100)^{\gamma} * (T_{\max} - T_{\min})$$

V0.9 :  
 $T_{\min} = 0.12$   
 $T_{\max} = 0.34$   
 $\gamma = 1.08$

La sélection reste probabiliste même avec une forte Intelligence. Cela évite un jeu parfaitement déterministe au niveau des décisions tout en conservant le déterminisme global grâce à la seed.

**Séparation conceptuelle essentielle :** Intelligence choisit mieux ; Passe, Tir, Technique, Vitesse et Physique exécutent mieux. Cette séparation doit rester un principe directeur lors de futures modifications.

## 9. Positionnement collectif, formation, bloc et appels

La seule formation disponible est aujourd'hui le 4-3-3. Elle est représentée comme un graphe de 11 slots : GK, LB, LCB, RCB, RB, CDM, LCM, RCM, LW, ST, RW.

Les ancres ne sont plus des positions fixes. Le moteur transforme collectivement la forme selon la possession et le ballon : translation longitudinale, largeur, profondeur, suivi latéral et hauteur de bloc.

| État collectif            | Effet actuel                                                                    |
|---------------------------|---------------------------------------------------------------------------------|
| Avec possession           | Bloc plus haut et plus large, davantage de profondeur.                          |
| Sans possession           | Bloc plus bas et plus compact.                                                  |
| Ballon sur un côté        | Le bloc suit latéralement la zone du ballon.                                    |
| Transition                | La réorganisation n'est pas instantanée, ce qui laisse de l'espace aux contres. |
| Ailiers proches du corner | Pénalité du dribble droit, bonus aux relais intérieurs/centres.                 |

Les appels sont des intentions persistantes runTarget/runUntil. Ils durent plusieurs secondes logiques, sont plus fréquents pour les joueurs offensifs et deviennent des cibles de passe plus attractives.

Le hors-jeu est pris en compte dans les appels et dans l'utility de passe. Les joueurs à forte Intelligence ajustent mieux leur position à la ligne du deuxième dernier défenseur.

## 10. Rôles, tactiques, synergies et hors-poste

### 10.1 Rôles individuels

| Rôle      | Effet principal                                                  |
|-----------|------------------------------------------------------------------|
| DEFENSIVE | Positionnement plus prudent, moins de risque.                    |
| NORMAL    | Comportement neutre.                                             |
| OFFENSIVE | Plus d'intentions vers l'avant et d'appels.                      |
| CREATOR   | Favorise les options progressives et la création.                |
| PRESSING  | Pressing plus agressif et coût énergétique légèrement supérieur. |

Les rôles ne sont pas des IA séparées. Ils ajustent les poids et offsets d'un agent commun. Les derniers Monte-Carlo montrent un delta global faible des rôles, ce qui est sain : un rôle change le style sans garantir automatiquement une victoire.

## 10.2 Tactiques collectives

- blockHeight : LOW / NORMAL / HIGH.
- buildUp : SHORT / BALANCED / DIRECT.

## 10.3 Synergie

Chaque slot possède des voisins tactiques. Chaque voisin actif de même nationalité ajoute actuellement +2 Intelligence, avec un cap à +6. La synergie est donc locale dans le graphe de formation, pas globale à toute l'équipe.

## 10.4 Hors-poste

Les positions naturelles et alternatives valent compatibilité 1. Une matrice règle les compatibilités secondaires. Le malus touche surtout Intelligence, puis légèrement Passe, Technique et Tir. Vitesse et Physique restent beaucoup plus stables.

```
Intelligence_effective =  
    Intelligence * compatibilité  
    + bonus_synergie  
    - pénalité_fatigue
```

# 11. Mouvement, fatigue, duels et états temporaires

La vitesse de déplacement est une interpolation entre minSpeedPerLogicalSecond et maxSpeedPerLogicalSecond selon la stat Speed. Le porteur est ralenti selon sa Technique, mais les joueurs peuvent désormais se déplacer rapidement par rapport aux premières versions.

La fatigue est intra-match uniquement. Elle réduit progressivement les stats effectives, avec un plancher configurable. Elle est totalement remise à zéro entre deux matchs au niveau du game design.

| Mécanique           | Implémentation                                                      |
|---------------------|---------------------------------------------------------------------|
| Fatigue             | Dépense liée au déplacement, rôle Pressing légèrement plus coûteux. |
| Multiplicateur stat | Plancher 0,82 en V0.9, courbe sqrt de l'énergie.                    |
| Duel                | Déclenchement à très courte distance.                               |
| Duel gagné          | Transfert physique de possession au point de contact.               |
| Duel perdu          | stunnedUntil 0,8 à 1,8 s, vitesse fortement réduite.                |

Le stun après duel a été introduit pour empêcher les boucles absurdes où deux joueurs se reprenaient immédiatement le ballon plusieurs fois.

# 12. Physique du ballon, passes et contrôles

La V0.9 utilise plusieurs états du ballon : CONTROLLED, LOOSE, DEAD et un ancien type TRANSIT encore présent dans les types internes. Les passes et tirs importants utilisent désormais surtout une balle libre avec position, vitesse et décélération.

| Propriété                              | Rôle                                            |
|----------------------------------------|-------------------------------------------------|
| pos                                    | Position continue normalisée sur le terrain.    |
| velocity                               | Vecteur vitesse.                                |
| deceleration                           | Perte de vitesse progressive.                   |
| sourceTeamIndex / actorId              | Origine de la trajectoire.                      |
| intendedReceiverId                     | Cible prévue, sans possession automatique.      |
| lastTouchTeamIndex / lastTouchPlayerId | Détermine sorties/corners/6 m.                  |
| kind                                   | PASS / SHOT / REBOUND / DEFLECTION / SET_PIECE. |

Une passe ne possède plus un booléen 'réussie' décidé au départ. La qualité de Passe détermine la direction, le dosage et l'erreur. Le ballon continue sa trajectoire, ralentit, peut être intercepté et doit réellement entrer dans le rayon de contrôle d'un joueur.

Le receveur prévu bénéficie d'une priorité comportementale mais pas d'un droit magique sur le ballon. Les joueurs courent vers le point d'arrêt prévisionnel.

## 12.1 Première touche

La probabilité de contrôle dépend de Technique, Passe, Intelligence, Physique et de la vitesse du ballon. En cas de mauvais contrôle, la balle est freinée et légèrement déviée mais reste vivante.

## 12.2 Passes au gardien

Les back-passes sont fortement découragées hors pression. Lorsqu'elles existent, la cible est placée quelques mètres devant le gardien, vers le terrain, afin d'éviter qu'une remise soit calibrée comme un tir vers sa propre ligne.

## 12.3 Passes arrière

L'utility pénalise les passes négatives, particulièrement les grosses remises vers l'arrière lorsque le porteur n'est pas pressé. Le jeu conserve toutefois les passes de soutien et les sorties de pression.

# 13. Tirs, gardiens, buts et rebonds

Depuis la V0.5, le résultat d'un tir n'est plus pré-tiré. Le moteur calcule une direction, une dispersion et une vélocité à partir de Shooting/Technique, puis simule la trajectoire.

**Invariant physique** : un but n'existe que si la balle franchit physiquement la ligne de but entre goalMouthMinY=0,42 et goalMouthMaxY=0,58.

Le gardien doit physiquement atteindre la trajectoire. Il anticipe les tirs et peut capter ou repousser s'il se place à portée. Les défenseurs peuvent également intercepter/contrer une trajectoire lorsqu'ils sont réellement sur son chemin.

## 13.1 Gardien comme agent spécifique

- Ancre et progression beaucoup plus conservatrices que le reste du bloc.
- Faible suivi latéral du ballon.
- Sortie sur balle libre seulement si la zone est dangereuse et s'il a un avantage de distance suffisant sur les défenseurs.
- Exception : anticipation d'une passe volontaire d'un coéquipier.
- Rayons distincts de capture, parade et contrôle d'une back-pass.

Visuellement, le gardien est rendu différemment des joueurs de champ dans le viewer, sous forme de losange avec contour spécifique.

# 14. Règles de football et coups de pied arrêtés

Les frontières du terrain sont physiques. Lorsqu'une balle franchit une ligne, le moteur détermine le point exact de sortie et la dernière équipe ayant touché le ballon.

| Événement  | Règle V0.9                                                            |
|------------|-----------------------------------------------------------------------|
| Touche     | Dernière touche adverse -> remise latérale.                           |
| Corner     | Défenseur/gardien touche en dernier avant sa ligne de but, hors cage. |
| Six mètres | Attaquant touche en dernier avant la ligne de but, hors cage.         |
| But        | Franchissement de ligne entre les poteaux.                            |
| Coup franc | Après faute ou hors-jeu.                                              |
| Penalty    | Faute dans la surface simplifiée.                                     |
| Mur        | 3 à 5 défenseurs sur coup franc dangereux.                            |

Un restart crée un état DEAD BALL, positionne le ballon, choisit un tireur, repositionne les joueurs et attend resumeAt avant la remise en jeu. Les arrêts alimentent le temps additionnel.

## 14.1 Positionnement sur coup d'envoi

Chaque joueur est replacé dans sa propre moitié. L'équipe qui ne donne pas le coup d'envoi est également maintenue hors du rond central. Le ST, ou à défaut un joueur de champ, est placé au centre pour engager.

## 14.2 Penalty

Le tireur est choisi parmi les joueurs de champ selon Shooting. Le gardien défenseur est replacé sur sa ligne. Le gardien de l'équipe qui tire reste dans son propre but. Les autres joueurs sont placés hors de la zone immédiate.

# 15. Hors-jeu, fautes, cartons, blessures et remplacements

## 15.1 Hors-jeu

Le moteur compare le receveur à la balle et au deuxième dernier adversaire. Une passe vers un joueur hors-jeu entraîne un coup franc indirect. L'Intelligence réduit aussi les dépassements de ligne lors des appels.

## 15.2 Fautes et cartons

- Fautes générées lors de certains duels ratés.
- Carton jaune possible selon sévérité.
- Deux jaunes -> rouge.
- Rouge direct possible.
- Un rouge crée une suspension d'un match dans le modèle produit.
- Pas d'accumulation de jaunes entre les matchs à ce stade.

## 15.3 Blessures

Une blessure peut apparaître après contact, avec influence de fatigue et Physical. La notification d'après-match prévoit actuellement une indisponibilité d'un match.

## 15.4 Remplacements

Maximum cinq. Les rotations automatiques utilisent des fenêtres planifiées autour de 62', 72' et 82', plus des seuils d'énergie et la gestion des blessures/cartons. Le système de remplacement sélectionne un joueur du banc compatible avec le poste.

**Limite actuelle :** le coach ne dispose pas encore d'un véritable éditeur de règles de remplacement pré-match. Les règles sont principalement codées/configurées côté moteur.

# 16. Replay et viewer 2D

Le replay est un artefact produit par le moteur. Il contient les métadonnées joueurs, les événements, les frames, l'état du ballon, l'énergie et l'horloge.

- Frame interval : 0,16 s logique.
- Interpolation côté client pour animation fluide.
- Le client ne recalcule aucune décision.
- Chronomètre 0-90 + temps additionnel.
- Score recalculé à partir des événements jusqu'au temps courant.
- Replay seekable par timeline et boutons temporels.
- Commentaires cliquables pour revenir au timestamp d'un événement.

Le repère interne du moteur reste horizontal HOME -> droite. Le viewer pivote la présentation afin que l'équipe représentée apparaisse en bas et attaque vers le haut.

## 17. Interface actuelle

### 17.1 Écran principal

- Seed déterministe éditable.
- Bouton de simulation.
- Nouvelle seed.
- Choix de couleur des deux équipes parmi 8 palettes.
- Taille du terrain ajustable.
- Scoreboard au-dessus du terrain.
- Terrain Canvas vertical.
- Commentaires à droite.
- Statistiques détaillées sous le terrain.
- Notifications après-match.

### 17.2 Contrôles du replay

La branche V0.9 intègre l'évolution du viewer issue de V0.8 : lecture, pause, vitesses remappées et navigation temporelle avant/arrière. Les commentaires du match sont des points d'entrée vers le replay.

- Vitesses visuelles : x0,5 / x1 / x2 / x4.
- Sauts : -10/-5/-2/-1 s et +1/+2/+5/+10 s.
- Timeline glissable.
- Clic sur un commentaire -> seek au timestamp et pause.

### 17.3 Événements visuels

Les événements importants possèdent des overlays temporaires. Le but a un traitement plus iconique. Les arrêts de jeu sont conçus pour rester visibles même à vitesse accélérée, sans faire dépendre la simulation de l'interface.

### 17.4 Rendu joueurs

- Joueurs de champ : formes géométriques simples avec numéro et nom court.
- Gardien : forme distincte / losange et contour spécifique.
- Ballon : blanc.
- Énergie : petite barre visuelle.
- Équipe HOME : bleu par défaut ; AWAY : rouge par défaut.

## 18. Balance Lab, Monte-Carlo, heatmaps et micro-benchmarks

Le Balance Lab est une deuxième application autour du moteur. Il exécute les matchs avec `recordReplay=false`, agrège les résultats et sert à mesurer le système plutôt qu'à le regarder.

### 18.1 Monte-Carlo

- 10 à 300 matchs par batch via l'interface.
- Même seedPrefix pour comparer des expériences appariées.
- Baseline sur les équipes de démonstration.
- Boost +10 d'une seule stat sur le onze domicile.
- Comparaison rôles configurés vs tous NORMAL.
- Erreur standard appariée du delta de buts.

### 18.2 Métriques de jeu

- Buts, tirs, cadrés, conversion.
- Passes tentées/réussies, passes arrière, back-passes GK.
- Dribbles/progressions, appels, duels.

- Transitions, récupérations, possession.
- Hors-jeu, touches, corners, six mètres, coups francs, penalties.
- Arrêts gardien, fautes, cartons, changements.
- Énergie moyenne finale.
- Temps additionnel.

### 18.3 Analytics spatiales

La baseline peut échantillonner les positions chaque seconde logique. Les heatmaps utilisent une grille 8 x 12 et peuvent être filtrées par poste.

- Centre moyen du bloc.
- Centre avec/sans possession.
- Largeur avec/sans possession.
- Profondeur du bloc.
- Hauteur de ligne défensive.
- Nombre moyen de joueurs dans la moitié adverse.
- Amplitude et volatilité du centre de bloc.

### 18.4 Micro-benchmarks isolés

| Stat         | Benchmark direct                   |
|--------------|------------------------------------|
| Speed        | Courses gagnées contre vitesse 75. |
| Shooting     | Tirs cadrés.                       |
| Passing      | Passes contrôlables.               |
| Physical     | Duels défensifs gagnés.            |
| Technique    | Premiers contrôles réussis.        |
| Intelligence | Sélection de la meilleure utility. |

Ces micro-tests sont essentiels : le delta de buts d'un match complet est chaotique dès la première divergence. Une statistique doit d'abord être monotone dans son domaine direct ; le Monte-Carlo global mesure ensuite son effet émergent.

## 19. État quantitatif observé en V0.9

Snapshot fourni lors du dernier test V0.9 avant l'abandon de la V0.10. Ces valeurs sont indicatives et dépendent des équipes d'élite utilisées, du nombre de runs et du seedPrefix.

| Indicateur                       | Valeur observée       |
|----------------------------------|-----------------------|
| Buts / match                     | 2,84                  |
| Victoire domicile                | 41 %                  |
| Nuls                             | 27 %                  |
| Tirs                             | 9,73 HOME / 8,35 AWAY |
| Passes                           | 62,09 / 61,01         |
| Réussite passes                  | 66,65 % / 63,65 %     |
| Conversion tirs                  | 15,83 % / 15,57 %     |
| Possession                       | 51,35 % / 48,65 %     |
| Énergie titulaires               | 58,16 % / 57,71 %     |
| Centre bloc possession / défense | 46 % / 43 %           |
| Amplitude bloc                   | 53 % du terrain       |
| Volatilité bloc                  | 13 %                  |
| Corners                          | 0,31 / 0,31           |
| Touches                          | 0,02 / 0,04           |

Les sorties de balle latérales restent nettement trop rares par rapport au football réel. Les corners restent également faibles. C'est une limite comportementale du style de jeu actuel, plus qu'une absence de règle.

### 19.1 Sensibilité globale observée

| Stat boostée +10 | Delta buts    | Lecture                         |
|------------------|---------------|---------------------------------|
| Intelligence     | +0,58 ± 0,248 | Signal global net sur ce batch. |
| Shooting         | +0,57 ± 0,206 | Signal global net sur ce batch. |

|           |               |                           |
|-----------|---------------|---------------------------|
| Physical  | +0,22 ± 0,177 | Bruit / signal modéré.    |
| Technique | +0,16 ± 0,226 | Bruit.                    |
| Speed     | -0,02 ± 0,221 | Négatif non significatif. |
| Passing   | -0,07 ± 0,246 | Négatif non significatif. |

Les micro-benchmarks de ce même test étaient tous positifs : Speed +14,32 pts, Shooting +6,63 pts, Passing +10,35 pts, Physical +3,03 pts, Technique +3,49 pts et Intelligence +3,47 pts.

**Interprétation recommandée** : ne pas modifier V0.9 uniquement parce qu'un delta de buts global est légèrement négatif alors que l'erreur standard est plus grande que le delta et que le micro-benchmark direct est monotone.

## 20. Ce qui est solide / ce qui reste fragile

| Solide aujourd'hui                  | Encore fragile / à surveiller                         |
|-------------------------------------|-------------------------------------------------------|
| Séparation serveur / replay client  | Fréquence réaliste des touches et corners             |
| Déterminisme seedé                  | Seulement une formation                               |
| Utility AI générique                | Réalisme fin des comportements par poste              |
| Trajectoires physiques du ballon    | Simplifications de collisions et dimensions physiques |
| Buts physiques                      | Gardien encore heuristique                            |
| Contrôles / interceptions physiques | Set pieces très simplifiés tactiquement               |
| Bloc collectif dynamique            | Hors-jeu très rare avec joueurs élite                 |
| Fatigue / duel / stun               | Blessures et suspensions simplifiées                  |
| Analytics et micro-benchmarks       | Balance statistique dépend des équipes de démo        |
| Replay lisible et seekable          | UI encore prototype produit, pas écran final de jeu   |

## 21. Limites connues et dette technique

- engine.ts est devenu très volumineux : la modularité conceptuelle existe, mais le fichier central devrait être découpé avant une forte croissance.
- Un type TransitBall subsiste alors que la simulation moderne s'appuie principalement sur LooseBall ; il existe de la dette historique à nettoyer prudemment.
- Les règles ne reproduisent pas toutes les nuances IFAB : pas de main, avantage, faute détaillée, arbitre, double-contact, coup franc indirect complet, etc.
- Le terrain est normalisé 0..1 et ne modélise pas explicitement mètres, accélération, orientation corporelle ou inertie du joueur.
- Le gardien est un agent spécialisé heuristique, pas un modèle complet d'angle de fermeture, plongeon et prise de balle.
- Pas de pied faible, taille, détente détaillée ou orientation dans le moteur malgré leur présence possible dans les données sources.
- Les joueurs n'ont pas encore de mémoire comportementale longue ni d'apprentissage.
- Les substitutions tactiques sont encore automatisées par règles moteur, pas construites par l'utilisateur.
- Le système de stratégie ne propose que deux instructions collectives.
- Aucune persistance d'une compétition, d'un historique de match ou d'un état de saison.

### 21.1 Risque majeur futur : couplage

La principale menace pour le projet serait de laisser l'écran d'équipe, la collection, l'économie et le matchmaking appeler directement les structures internes du moteur. Il faut maintenir une frontière nette : ils construisent un MatchInput versionné ; le moteur renvoie un MatchOutput versionné.

## 22. Architecture modulaire recommandée pour la suite

|                                                 |                                                                                                                     |
|-------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|
| packages/<br>shared/<br>player-data/<br>roster/ | types, schemas, enums, versioning<br>import, repository, queries joueurs<br>collection possédée, squads, validation |
|-------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|

|                  |                                              |
|------------------|----------------------------------------------|
| tactics/         | formations, slots, roles, substitution plans |
| match-engine/    | simulation pure V0.9+                        |
| match-analytics/ | Monte-Carlo, micro-benchmarks, diagnostics   |
| replay/          | format, compression, migrations              |
| competitions/    | règles de tournoi/ligue                      |
| economy/         | packs, monnaie, récompenses (plus tard)      |

|         |                                           |
|---------|-------------------------------------------|
| apps/   |                                           |
| web/    | Next.js UI                                |
| api/    | API métier si séparée                     |
| worker/ | simulation async / batch à grande échelle |

Il n'est pas obligatoire de créer immédiatement un monorepo multi-package. Mais les frontières doivent être conçues comme si ces packages existaient.

| Module futur | Ne doit connaître que                                        |
|--------------|--------------------------------------------------------------|
| Écran équipe | PlayerSummary, Squad, FormationDefinition, TacticsDefinition |
| API match    | MatchInput / MatchOutput                                     |
| Viewer       | ReplaySchema uniquement                                      |
| Collection   | PlayerId et métadonnées de propriété                         |
| PvP async    | Snapshot immutable de squad/tactique                         |
| Analytics    | Match engine public + diagnostics versionnés                 |

**Conseil architectural :** introduire des schémas runtime (par exemple Zod) aux frontières API et persistance lorsque l'application devient multi-module. TypeScript seul ne protège pas les données externes.

## 23. Prochaine étape recommandée : équipe et tactiques

La prochaine étape la plus logique est bien l'écran de gestion d'équipe. Il transforme enfin le simulateur en jeu, sans modifier son coeur.

### 23.1 Module Squad Builder

1. Charger la collection de joueurs possédés.
2. Recherche par nom.
3. Filtres : poste, nationalité, notes, fourchette d'overall.
4. Afficher une carte joueur compacte avec les six stats.
5. Drag & drop vers les slots d'une formation.
6. Afficher immédiatement la compatibilité hors-poste.
7. Afficher les liens de synergie entre slots voisins.
8. Gérer le banc.
9. Valider qu'un joueur n'est utilisé qu'une fois.
10. Sauvegarder plusieurs compositions.

### 23.2 Module tactique

11. Choisir la formation parmi un catalogue.
12. Attribuer un rôle à chaque slot.
13. Choisir hauteur de bloc et construction.
14. Configurer les remplacements pré-match.
15. Afficher un résumé tactique calculé : compatibilité moyenne, synergie, profils de vitesse/intelligence, etc.
16. Créer un TeamSelection immutable envoyé au moteur.

### 23.3 Ce qu'il ne faut pas faire

- Ne pas importer engine.ts dans les composants de composition.
- Ne pas dupliquer les règles de compatibilité côté UI : exposer une fonction/service partagé.
- Ne pas stocker une composition uniquement comme positions de pixels ; stocker formationId + slotId + playerId.
- Ne pas rendre le résultat du match dépendant d'un état React non sérialisable.

## 24. Roadmap produit complète

| Phase | Livrable                                        | Priorité     |
|-------|-------------------------------------------------|--------------|
| A     | Squad Builder + recherche joueurs + banc        | Très haute   |
| B     | Catalogue de formations + rôles + tactiques     | Très haute   |
| C     | Sauvegarde locale/DB de plusieurs équipes       | Très haute   |
| D     | Lancement d'un match depuis la composition      | Très haute   |
| E     | Collection / propriété des joueurs              | Haute        |
| F     | Bots avec squads variées et difficultés         | Haute        |
| G     | PvP asynchrone par snapshots                    | Haute        |
| H     | Compétitions simples / classement / récompenses | Moyenne      |
| I     | Économie / acquisition de joueurs               | Moyenne      |
| J     | Historique, statistiques joueurs/équipes        | Moyenne      |
| K     | Polish UI, onboarding, responsive final         | Moyenne      |
| L     | Déploiement, observabilité, anti-triche         | Avant public |

## 25. Idées d'évolution du moteur à plus long terme

Ces idées ne sont pas nécessaires pour la prochaine version. Elles doivent être considérées uniquement lorsque le jeu autour du moteur est déjà amusant.

- Plusieurs formations : 4-2-3-1, 4-4-2, 3-5-2, 4-1-2-1-2, etc.
- Instructions collectives supplémentaires : tempo, largeur, pressing, ligne défensive, centres, risque de passe.
- Rôles dépendants du poste plutôt que cinq rôles universels.
- Plan de remplacement configurable : minute, énergie, carton, score, position.
- Orientation corporelle / pied préféré / pied faible.
- Taille et détente pour jeu aérien.
- Centres et duels aériens explicites.
- Pressing coordonné à plusieurs joueurs.
- Notion de couverture et marquage individuel/zone plus avancée.
- Transitions tactiques configurables.
- Qualité de décision contextuelle distincte offensive/défensive.
- Personnalités d'agents ou traits cachés.
- Mémoire courte : dernier duel, confiance, répétition d'un pattern.
- Agents apprenants hors production pour découvrir des politiques, puis distillation dans une IA interprétable.
- Simulation multi-fidélité : mode rapide sans replay pour compétitions massives.

### 25.1 IA / agents : conseil de recherche

Le modèle actuel est une excellente base expérimentale pour étudier des agents. Une évolution intéressante serait de conserver l'utility AI comme policy explicable de production et d'utiliser RL / imitation / optimisation hors-ligne uniquement comme outil de calibration des poids. Cela préserverait la reproductibilité et l'explicabilité tout en permettant de découvrir des comportements.

**À éviter au début** : un LLM par joueur pendant chaque tick. Le coût, la latence, la non-déterminisme et la difficulté de validation seraient incompatibles avec l'objectif navigateur et le Monte-Carlo massif.

## 26. Stratégie de tests et qualité

### 26.1 Tests unitaires recommandés

- Même MatchInput + seed = même MatchOutput.
- Aucun NaN / Infinity.
- Aucun joueur actif hors terrain sauf états explicitement permis.

- But uniquement après franchissement physique de ligne.
- Corner/six mètres/touche déterminés par dernière touche.
- Kickoff : chaque équipe dans sa moitié.
- Penalty : chaque gardien devant son propre but.
- Un joueur expulsé ne revient pas.
- Maximum 5 remplacements.
- Pas de joueur dupliqué dans une équipe.
- Balle libre près d'une ligne toujours récupérable ou déclarée sortie.

## 26.2 Tests statistiques

- Micro-benchmark monotone pour chacune des 6 stats.
- Distribution des buts par niveau moyen d'équipe.
- Taux de tirs/cadrés/conversion.
- Pass completion par distance.
- Fatigue finale par rôle et Physical.
- Fréquence des touches/corners/hors-jeu.
- Win rate par formation lorsque plusieurs formations existent.
- Impact des rôles ne devant pas dominer les notes joueurs.

## 26.3 Tests de replay

- Dernière frame cohérente avec score final.
- Événement GOAL aligné avec franchissement visible.
- Seek et interpolation sans téléportation majeure.
- Commentaires cliquables vers le bon timestamp.
- Les vitesses d'affichage ne changent jamais le résultat.

# 27. Handoff à un futur LLM / développeur

**Règle n°1** : La branche conceptuelle de référence est V0.9. Ne pas appliquer les changements de V0.10 sans nouvelle décision explicite.

## 27.1 Ordre de lecture conseillé

17. Game Design initial : fut\_manager\_game\_design\_v1.pdf.
18. Présent rapport V0.9.
19. README.md de la V0.9.
20. lib/game/types.ts.
21. lib/game/config.ts.
22. lib/game/formations.ts et compatibility.ts.
23. lib/game/engine.ts.
24. PitchCanvas.tsx et MatchSimulator.tsx.
25. Route Monte-Carlo + micro-benchmarks.
26. Changelogs V0.2 -> V0.9 pour comprendre l'origine de certains choix.

## 27.2 Commandes de validation

```
npm install
npm run typecheck
npm run build
npm run dev
```

## 27.3 Questions à poser avant toute modification moteur

27. Est-ce un bug réel ou un artefact visuel du replay ?
28. La correction doit-elle être faite dans le moteur, le replay ou le viewer ?
29. Est-ce que cela casse le déterminisme ?

- 30. Est-ce que cela change un micro-benchmark direct ?
- 31. Est-ce que l'effet Monte-Carlo est statistiquement supérieur au bruit ?
- 32. Peut-on rendre le coefficient configurable plutôt que hardcodé ?
- 33. Le comportement peut-il être testé automatiquement ?

## **27.4 Invariant de modularité**

Toute nouvelle fonctionnalité produit doit être conçue pour pouvoir remplacer son UI ou son stockage sans modifier le moteur. Le moteur doit pouvoir être exécuté depuis un test, un worker, une API ou un script analytique sans dépendre de Next.js.

## Annexe A. Fichiers importants de la V0.9

| Fichier                                | À quoi il sert                                           |
|----------------------------------------|----------------------------------------------------------|
| lib/game/engine.ts                     | Simulation complète. Fichier central à découper à terme. |
| lib/game/config.ts                     | Tous les coefficients principaux.                        |
| lib/game/types.ts                      | Contrats publics et types replay/stats.                  |
| lib/game/formations.ts                 | 4-3-3, ancrés, voisinages.                               |
| lib/game/compatibility.ts              | Matrice hors-poste et stats effectives.                  |
| lib/game/rng.ts                        | Aléatoire déterministe.                                  |
| lib/game/sample-teams.ts               | Équipes de démonstration utilisées par UI/analytics.     |
| lib/data/load-players.ts               | Charge le CSV joueurs.                                   |
| app/api/matches/simulate/route.ts      | Endpoint serveur de match.                               |
| components/PitchCanvas.tsx             | Viewer replay Canvas.                                    |
| components/MatchSimulator.tsx          | UI principale du simulateur.                             |
| app/api/analytics/monte-carlo/route.ts | Runner batch et expériences.                             |
| lib/analytics/micro-benchmarks.ts      | Tests directs des stats.                                 |
| components/AnalyticsDashboard.tsx      | Balance Lab.                                             |

## Annexe B. Invariants à ne pas casser

- Le serveur décide ; le viewer affiche.
- Seed identique + entrée identique = résultat identique.
- Pas de score tiré avant le match.
- Pas de but probabiliste : franchissement physique obligatoire.
- Un receveur doit physiquement atteindre/contrôler la balle.
- Intelligence = qualité de décision, pas substitut universel aux autres stats.
- Hors-poste pénalise surtout Intelligence.
- Synergie augmente Intelligence et reste locale aux voisins tactiques.
- Fatigue intra-match, reset entre matchs.
- Les couleurs d'équipe sont purement visuelles.
- Les analytics headless ne doivent pas dépendre du Canvas.
- Une modification d'équilibrage doit être confrontée aux micro-benchmarks ET au Monte-Carlo.

## Annexe C. Paramètres principaux de configuration V0.9

| Groupe            | Valeurs / remarques                                                   |
|-------------------|-----------------------------------------------------------------------|
| Temps             | 90 min affichées, 360 s logiques, physics 0,2 s, ball substep 0,04 s. |
| Décision          | T min 0,12 ; T max 0,34 ; gamma 1,08.                                 |
| Synergie          | +2 INT par voisin même nationalité, cap +6.                           |
| Fatigue           | minStatMultiplier 0,82 ; pressing x1,10.                              |
| Substitutions     | fenêtres 62/72/82, max 5.                                             |
| Mouvement         | vitesse min 0,03168 ; max 0,0792.                                     |
| Bloc              | attaque plus haute/large, défense plus basse/compacte.                |
| Appels            | 2,2 à 5,2 s ; chance de base 0,06 par décision.                       |
| GK                | baseProgress 0,042 ; possession 0,070 ; sortie max 0,19.              |
| Duel              | rayon 0,020 ; stun 0,8 à 1,8 s.                                       |
| Balle             | passDecel 0,12 ; shotDecel 0,025 ; control radius 0,010-0,017.        |
| Cage              | y 0,42 à 0,58.                                                        |
| Set pieces        | pauses 1,1 à 2,4 s, mur 3-5 joueurs.                                  |
| Temps additionnel | 72 % des arrêts compensés, cap logique par mi-temps.                  |
| Heatmap           | 8 colonnes x 12 lignes.                                               |

**Conclusion :** Le simulateur V0.9 est une base fonctionnelle et expérimentale solide. La meilleure manière de faire progresser le projet maintenant n'est plus d'ajouter continuellement des règles au moteur, mais de construire les modules de jeu autour de son contrat : collection, squad builder, formations, tactiques, sauvegarde, bots, PvP asynchrone et compétitions.